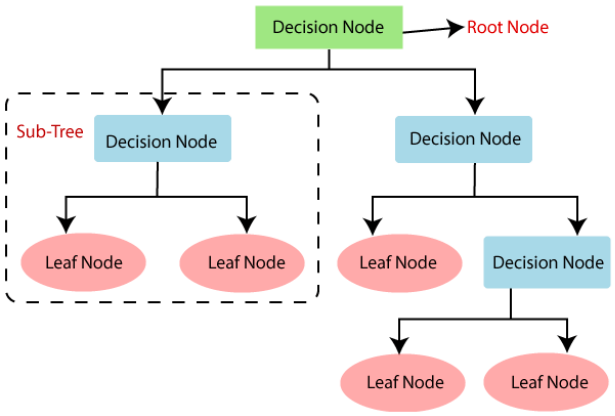
Week 8 : DECISION TREE and K-Means Clustering

DECISION TREE

A decision tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but it is mostly preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules,** and **each leaf node represents the outcome.**

- In a Decision tree, there are two nodes: the **Decision Node** and **the leaf node.** Decision nodes are used to make decisions and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the tests are performed based on features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the **CART algorithm,** which stands for **Classification and Regression Tree algorithm.**
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- The diagram explains the general structure of a decision tree:



Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

o Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
o The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

Root Node: Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

Leaf Node: Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

Splitting: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

Branch/Sub Tree: A tree formed by splitting the tree.

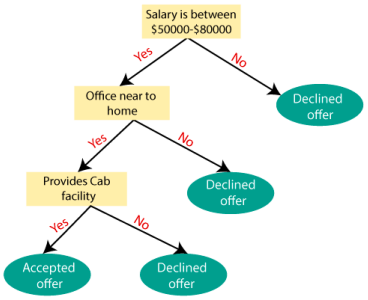
Pruning: Pruning is the process of removing the unwanted branches from the tree.

Parent/Child node: The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.
For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:
o **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
o **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**
o **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
o **Step-4:** Generate the decision tree node, which contains the best attribute.
o **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offers). Consider the diagram below:



Attribute Selection Measures
While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM.** By this measurement, we can easily select the best attribute for the nodes of the tree.
There are two popular techniques for ASM, which are:
o Information Gain
o Gini Index

1. Information Gain:
o Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
o It calculates how much information a feature provides us about a class.
o According to the value of information gain, we split the node and build the decision tree.
o A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)
Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:
Entropy(s)= -P(yes)log2 P(yes)- P(no) log2 P(no)
Where,
o S= Total number of samples
o P(yes)= probability of yes

Entropy-It is used for checking the impurity or uncertainty present in the data. Entropy is used to evaluate the quality of a split. When entropy is zero the sample is completely homogeneous, meaning that each instance belongs to the same class and entropy is one when the sample is equally divided between different classes.

Formula of Entropy –

$$Entropy = \sum_{i=1}^{C} -p_i * \log_2(p_i)$$

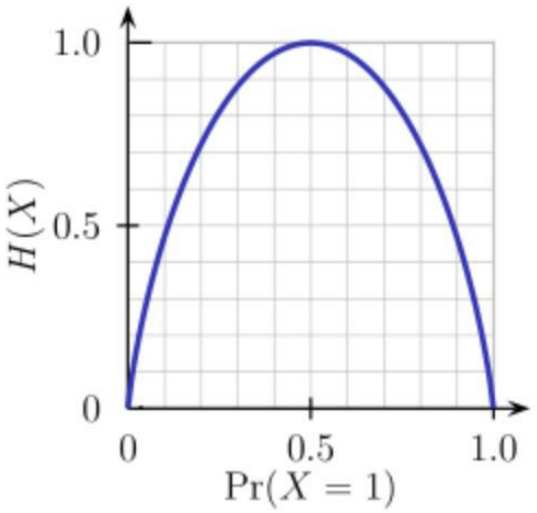
Information Gain- Information gain indicates how much information a particular feature/ variable give us about the final outcome.
Formula of information gain-

$$Gain(T, X) = Entropy(T) - Entropy(T, X)$$

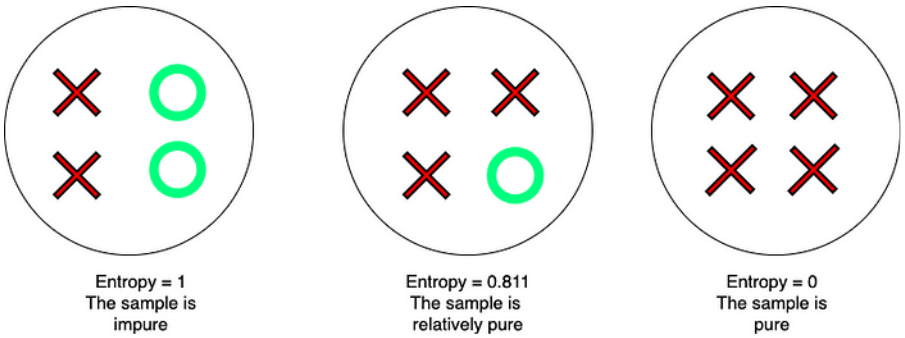
C4.5
C4.5 Decision Tree is a complicated Algorithm to understand. It does require a lot of background knowledge.
Information Entropy : Information Entropy is the measure of impurity in a given example. Let's elaborate *Shanon entropy or self information:*

$$-\Sigma_{i=1}^{n} P(x_i) \log_b(P(x_i))$$

Where b is the base.
For a binary, the base would be 2 and the plot of the function would be as follows for percentages of 1 in the dataset:



So if the Probability of an event approaches 1 or 0, the Information Entropy tends to 0 as the output is more or less predictable. In statistics, entropy measures the level of impurity(heterogeneity) in a dataset. A fully homogeneous dataset has an entropy of 0, whereas a skewed dataset has an entropy closer to 1 as shown in the figure below:



Information Gain : It is a parameter that is used to compute the change in entropy of a dataset before and after a transformation. Information Gain helps in feature selection. How it works is as follows. A feature in a dataset is chosen and based on the value of the feature, the dataset is split into multiple smaller datasets. The change in the overall entropy from that of the parent to the average entropy of all the new child datasets is computed to be the Information Gain.

$$Information\ Gain = Entropy_{parent} - \sum Entropy_{child}$$

The higher the Information Gain the more accurately the feature divides the dataset as the resultant dataset are more homogenous with lower entropies. This indicates the feature has split(classified) the dataset accurately. On the other hand, if the Information Gain is low, it implies that the resultant datasets are more or less as heterogeneous as the parent dataset and so the feature does not provide much value.

Steps in the algorithm:

1. Calculate the Entropy_information for the entire training dataset

2. Compute the entropy_info, information gain, and split_info and gain_ratio for each attribute in the training dataset

4. Choose the attribute for which gain_ratio is maximum as the best split attribute
5. The best split attribute is chosen as the root node
6. Next, the root node is split into subtrees as an outcome of test condition
7. Recursively apply the same operation for the subset of the training dataset until leaf node is derived and no more training instances are available

Exercises

1. Write a python function program to demonstrate the working of the decision tree based **C4.5** algorithms without using scikit-learn library. Use following data set for building the decision tree and apply this knowledge to classify a new sample.
The dataset has three attributes: Outlook (Sunny, Overcast, Rainy), Temperature, Humidity and Wind (Weak, Strong). The target attribute is Play Tennis (Yes/No).

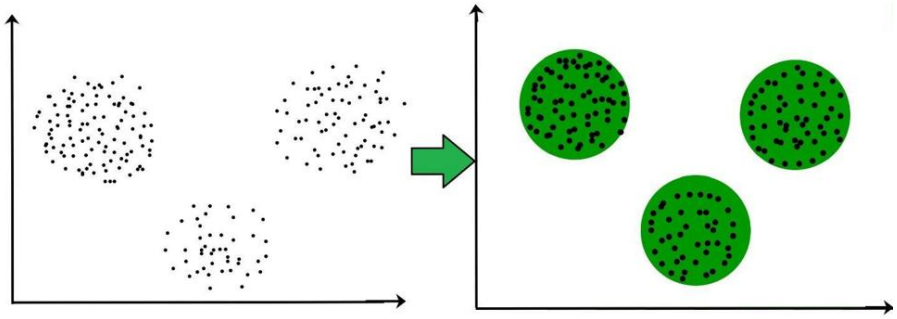
Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	85	85	Weak	No
2	Sunny	80	90	Strong	No
3	Overcast	83	78	Weak	Yes
4	Rain	70	96	Weak	Yes
5	Rain	68	80	Weak	Yes
6	Rain	65	70	Strong	No
7	Overcast	64	65	Strong	Yes
8	Sunny	72	95	Weak	No
9	Sunny	69	70	Weak	Yes
10	Rain	75	80	Weak	Yes
11	Sunny	75	70	Strong	Yes
12	Overcast	72	90	Strong	Yes
13	Overcast	81	75	Weak	Yes
14	Rain	71	80	Strong	No

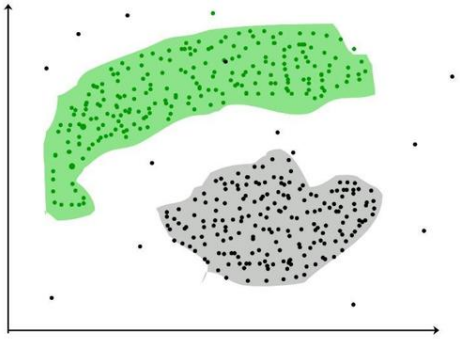
2. Write a Python function program to demonstrate the working of the decision tree based on C4.5 without and with using the scikit-learn library. Using the following dataset, apply the aforementioned algorithms. The attributes are Income (Low, Medium, High) and Credit (Good, Bad), and the target is Loan Approved (Yes/No).

K-Means Clustering

Introduction to Clustering: It is basically a type of unsupervised learning method. An unsupervised learning method is a method in which we draw references from datasets consisting of input data without labeled responses. Generally, it is used as a process to find meaningful structure, explanatory underlying processes, generative features, and groupings inherent in a set of examples.
Clustering is the task of dividing a population or data points into groups such that points in the same group are more similar to one another and dissimilar to points in other groups. It is basically a collection of objects on the basis of similarity and dissimilarity between them.

For example, the data points in the graph below clustered together can be classified into one single group. We can distinguish the clusters, and we can identify that there are 3 clusters in the picture below:





Why Clustering?
Clustering is very important as it determines the intrinsic grouping among the unlabelled data present. There are no criteria for good clustering. It depends on the user and what criteria they may use, which satisfy their need. For instance, we could be interested in finding representatives for homogeneous groups (data reduction), finding "natural clusters" and describing their unknown properties ("natural" data types), in finding useful and suitable groupings ("useful" data classes), or finding unusual data objects (outlier detection). This algorithm must make some assumptions that constitute the similarity of points, and each assumption makes different and equally valid clusters.

K-Means Clustering
K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning or data science. In this topic, we will learn what is K-means clustering algorithm, how the algorithm works, along with the Python implementation of K-means clustering.
The goal of clustering is to divide the population or set of data points into a number of groups so that the data points within each group are more comparable to one another and different from the data points within the other groups. It is essentially a grouping of things based on how similar and different they are to one another.
We are given a data set of items, with certain features, and values for these features (like a vector). The task is to categorize those items into groups. To achieve this, we will use the K-means algorithm, an unsupervised learning algorithm. 'K' in the name of the algorithm represents the number of groups/clusters we want to classify our items into.
(It will help if you think of items as points in an n-dimensional space). The algorithm will categorize the items into k groups or clusters of similarity. To calculate that similarity, we will use the Euclidean distance as a measurement.
The algorithm works as follows:
1. First, we randomly initialize k points, called means or cluster centroids.
2. We categorize each item to its closest mean and we update the mean's coordinates, which are the averages of the items categorized in that cluster so far.

3. We repeat the process for a given number of iterations and at the end, we have our clusters.
The "points" mentioned above are called means because they are the mean values of the items categorized in them. To initialize these means, we have a lot of options. An intuitive method is to initialize the means at random items in the data set. Another method is to initialize the means at random values between the boundaries of the data set (if for a feature *x,* the items have values in [0,3], we will initialize the means with values for *x* at [0,3]).

Exercises

Consider the following three variables for 20 different basketball players: points, assists, and rebounds. Perform k-means clustering manually with K=2, using Euclidean distance. Show the working for one iteration in your Lab Observation Book by using Euclidean distance. Mark the same on the graph.

points	assists	rebounds
18.0	3.0	15
19.0	4.0	14
14.0	5.0	10
14.0	4.0	8
11.0	7.0	14
20.0	8.0	13
28.0	7.0	9
30.0	6.0	5
31.0	9.0	4
35.0	12.0	11
33.0	14.0	6
25.0	9.0	5
25.0	4.0	3
27.0	3.0	8
29.0	4.0	12
30.0	12.0	7
19.0	15.0	6
23.0	11.0	5

Write a Python function (without using the scikit-learn library) to create a DataFrame containing the three variables (points, assists, and rebounds) for 20 different basketball players. Apply the K-means algorithm to identify clusters with K=1, 2, K=3, and K=4, using distance formulas such as Euclidean distance, Manhattan distance, and Minkowski distance. Perform the following tasks:

a. Create a scatter plot of the data points in blue.

b. Plot the clusters with data points in different colors for K=1, 2, 3, and 4 in separate graphs.

c. Create a plot showing the number of clusters on the x-axis and the Sum of Squared Errors(SSE) on the y-axis. Compute SSE for all iterations. Show the table of given data points against SSE for every iteration and use the total sum of SSE in the graph of K vs. SSE.

d. Show the optimal value of K using the Elbow method and mark the same in the graph.